

머신러닝

선형회귀 기초

Chapter 6



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

- 01 선형회귀의 이해
- 02 선형회귀의 기초 수식
- 03 최소자승법으로 선형회귀 풀기
- 04 경사하강법으로 선형회귀 풀기
- 05 선형회귀 성능 평가
- 06 코드로 선형회귀 구현

01 선형회귀의 이해



선형회귀의 개념

- 선형회귀(Linear Regression)

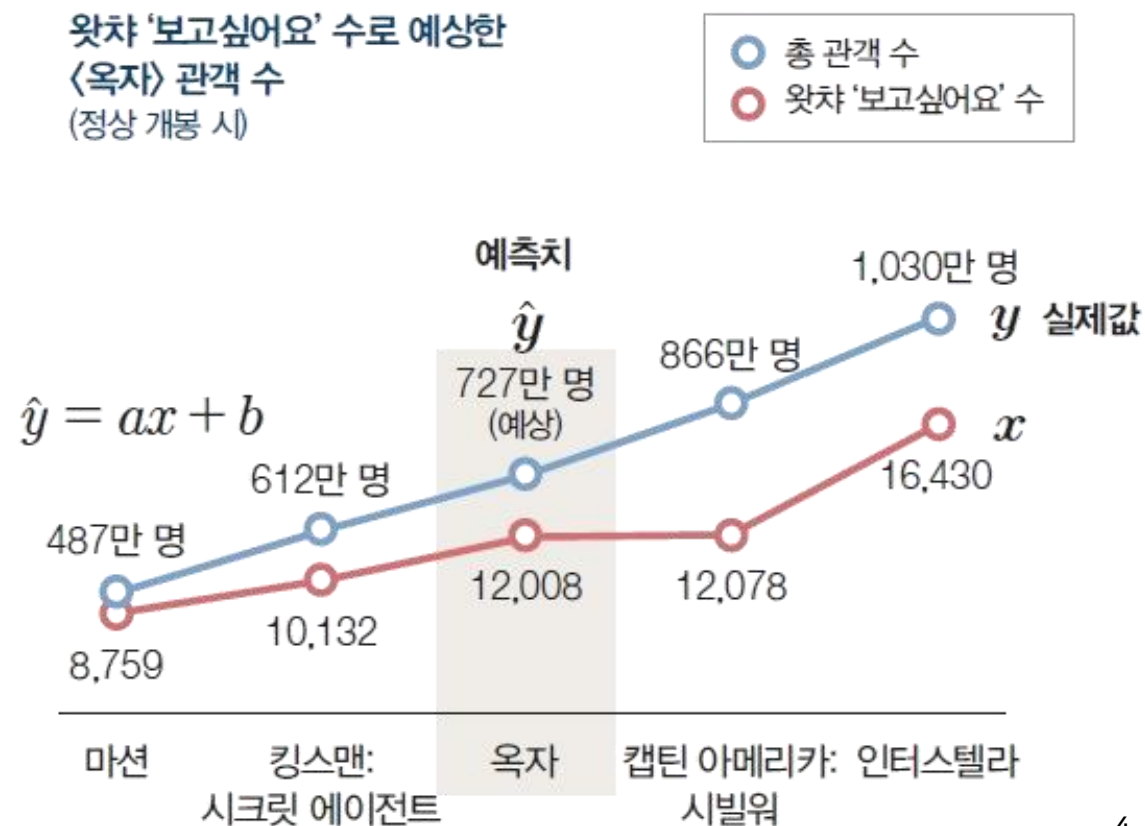
: 종속변수 y 와 한 개 이상의 독립변수 x 와의 선형 상관관계를 모델링하는 회귀분석 기법

- 지도학습 알고리즘

- 기존 데이터를 활용해 연속형 변수값을 예측

- $y = ax + b$ 꼴의 수식을 만들고 a 와 b 의 값을 찾아냄

- 앞으로 개봉할 영화 예상 관객 수 y 를 예측하는 문제





선형회귀의 개념

- 실제 관객 수를 y 로 표현하여 좌표평면 상에 나타냄

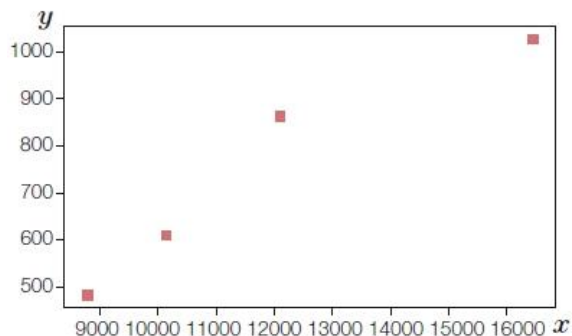
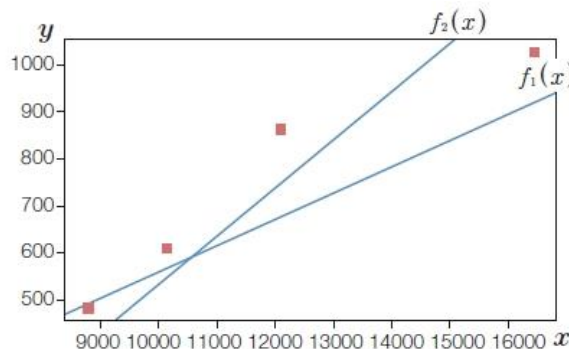


그림 7-2 좌표평면에 작성한 2개의 직선, $f_1(x)$ 와 $f_2(x)$



- 두 그래프 중 어떤 것이 기존 데이터를 '잘 표현하는가'
- 예측값이 실제값 대비 차이가 많이 나지 않는 그래프

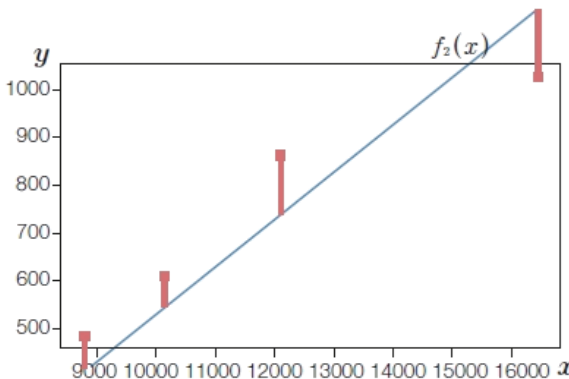
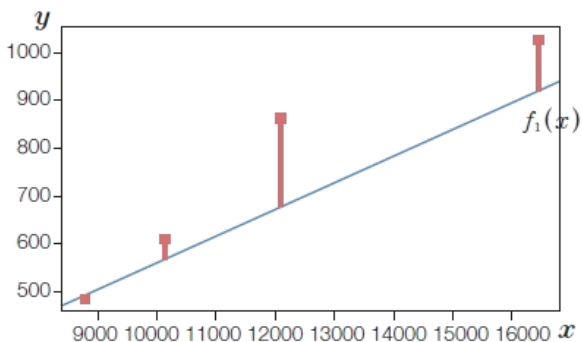


그림 7-3 $f_2(x)$ 가 $f_1(x)$ 보다 좀 더 데이터를 잘 표현하고 있다.



예측 함수와 실제값 간의 차이

- 예측 함수는 예측값과 실제값 간의 차이를 최소화하는 방향으로
- 데이터 n 개 중 i 번째 데이터의 y 값에 대한 실제값과 예측값($\hat{y} = w_1x + w_0$)의 차이
같은 의미 : $y = ax + b$
- 데이터가 5개 있을 때 5개 데이터의 오차의 합

$$\hat{y}^i - y^i \approx 0$$

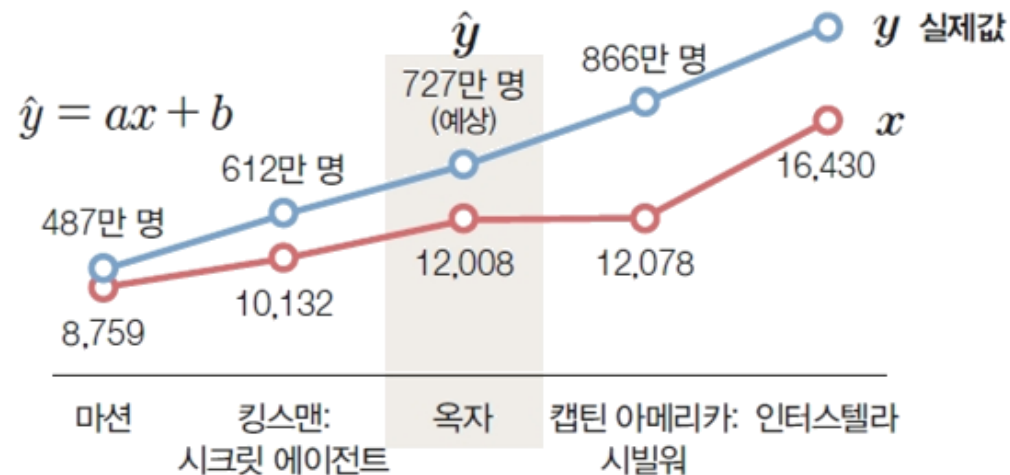
- 오차 값들이 음수와 양수로 나왔을 때 값들 간의 차이가 상쇄되어 0에 가까운 값으로 만들어야함

$$(\hat{y}^{(1)} - y^{(1)}) + (\hat{y}^{(2)} - y^{(2)}) + (\hat{y}^{(3)} - y^{(3)}) + (\hat{y}^{(4)} - y^{(4)}) + (\hat{y}^{(5)} - y^{(5)}) = \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) \approx 0$$

- 같은 식을 행렬로 표현

$$\hat{y} = \begin{bmatrix} w_1 \times 8759 + w_0 \\ w_1 \times 10132 + w_0 \\ w_1 \times 12078 + w_0 \\ w_1 \times 16430 + w_0 \end{bmatrix} \quad y = \begin{bmatrix} 487 \\ 612 \\ 866 \\ 1030 \end{bmatrix}$$

$$(\hat{y} - y)^2 = \begin{bmatrix} (w_1 \times 8759 + w_0 - 487)^2 \\ (w_1 \times 10132 + w_0 - 612)^2 \\ (w_1 \times 12078 + w_0 - 866)^2 \\ (w_1 \times 16430 + w_0 - 1030)^2 \end{bmatrix}$$





예측 함수와 실제값 간의 차이

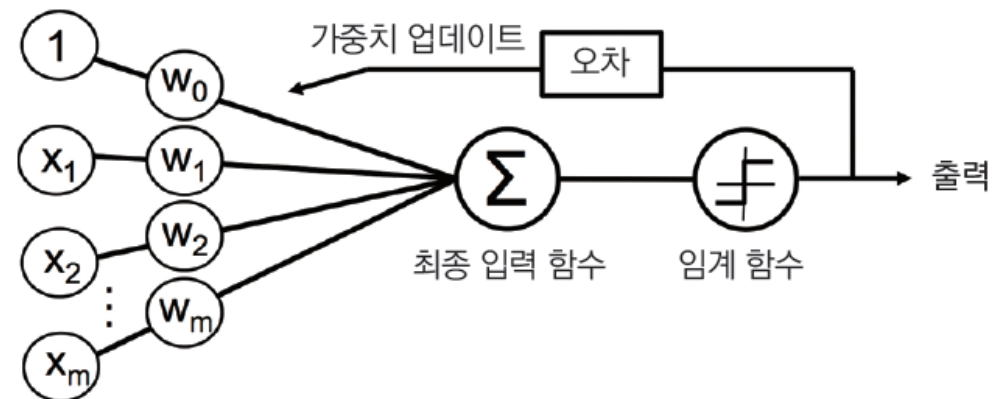
- 제곱 오차(square error) : $(\hat{y} - y)^2$ 로 예측값과 실제값의 제곱을 표시하여 오차를 나타냄

$$\hat{y} = w_1x + w_0$$

- 제곱 오차를 최소화하는 w_0 와 w_1 을 찾아야 함

$$\sum_{i=1}^n (w_1x^{(i)} + w_0 \times 1 - y^{(i)})^2 \approx 0$$

- 머신러닝에서는 아래와 같은 과정으로 w_0 와 w_1 를 찾음



02 선형회귀의 기초 수식



비용함수의 개념

- 가설함수(hypothesis function) : 예측값($\hat{y}$)을 생성하는 함수

$$f(x) = h_{\theta}(x) = \sum_{i=1}^n w_i x_i + w_0$$

- 함수 입력값은 x
- 함수에서 결정할 것은 θ =가중치(weight) 값인 w_n

- 비용함수(cost function) : 머신러닝에서 최소화해야 할 예측값과 실제값의 차이

$$\sum_{i=1}^n (w_i x_i + w_0 \times 1 - y_i)^2 \approx 0$$



비용함수

- 비용함수가 두 개의 가중치 값으로 결정됨

- 잔차의 제곱합(Error sum of squares) : 예측값인 가설함수와 실제값인 y 값 간의 차이를 제곱해서 모두 합함
 - 총 데이터는 m 개가 존재하고 각 데이터의 예측값과 실제값을 뺀 후 제곱한 값들을 모두 합한 값
- 손실함수(loss function) : 비용함수에서 잔차의 제곱합 부분

$$\sum_{i=1}^m (h_{\theta}(x^{(i)}) - (y^{(i)}))^2 = \sum_{i=1}^m (w_i x_i + w_0 \times 1 - y_i)^2 \quad (m = \text{row size})$$

- 평균 제곱 오차(mean squared error, MSE) : 잔차의 제곱합을 $2m$ 으로 나눈 값

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - (y^{(i)}))^2$$

Q: 왜 2를 나누는가?

A: 제곱항을 미분하면 2가 나오므로 $2 \times \frac{1}{2} = 1 \Rightarrow$ 상수는 사라짐

Q: 왜 미분을 하는가?

A: 비용함수의 기울기를 구하여 비용이 최소가 되는 가중치를 찾기 위함



비용함수

- 비용함수를 수식의 목적이 나타나도록 표기

$$\arg \min_{\theta} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- 비용함수 그래프

- x, y축에는 w_0 와 w_1 값
- z축에는 비용함수 값 J
- 가중치 값 설정에 따른 J 값을 볼 수 있음

Q: 왜 비용함수는 제곱하여 2차함수를 그리는가?

A: 오차를 제곱하면 항상 양수가 되고 미분이 가능한
2차 함수가 되어 최소값을 찾기 쉬워지기 때문

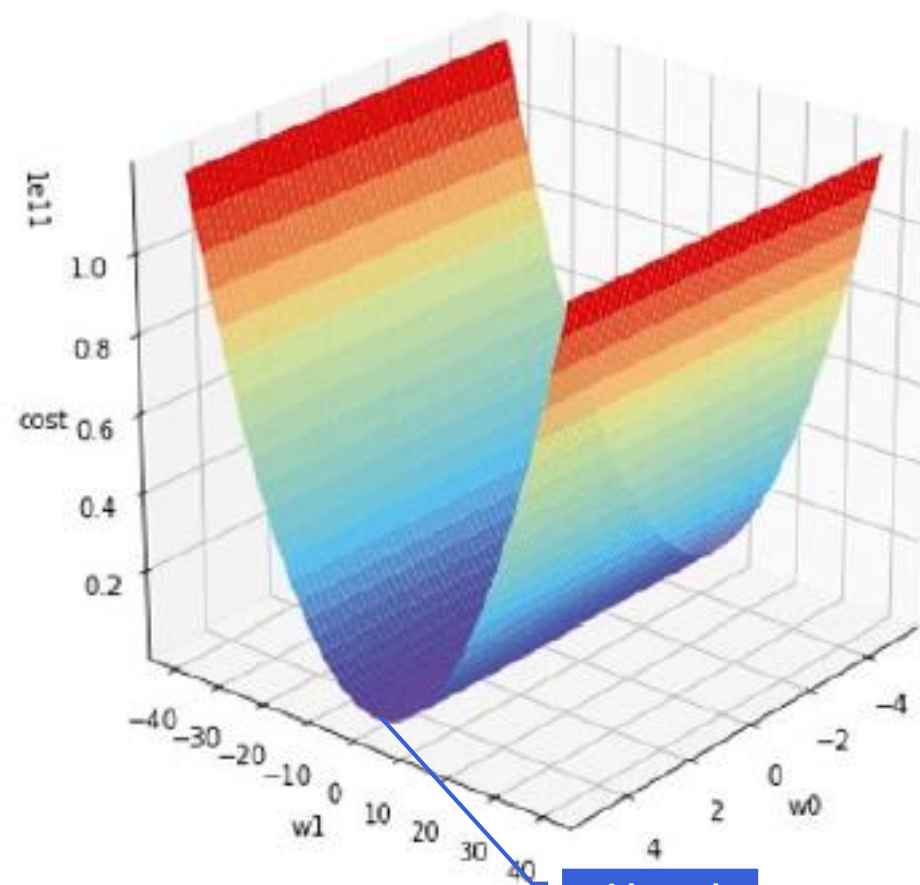


그림 7-4 비용함수 그래프

최소값

03

최소자승법으로 선형회귀 풀기



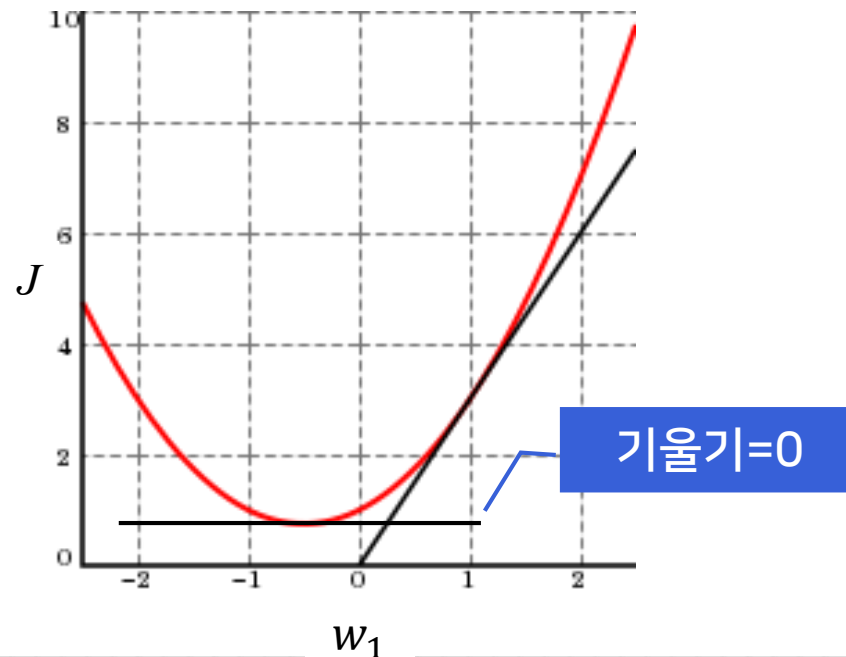
최소자승법(least square method)

- 최소자승법(least square method) : 선형대수의 표기법을 사용하여 방정식으로 선형회귀 문제를 푸는 방법

- 비용함수를 미분

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})^2$$

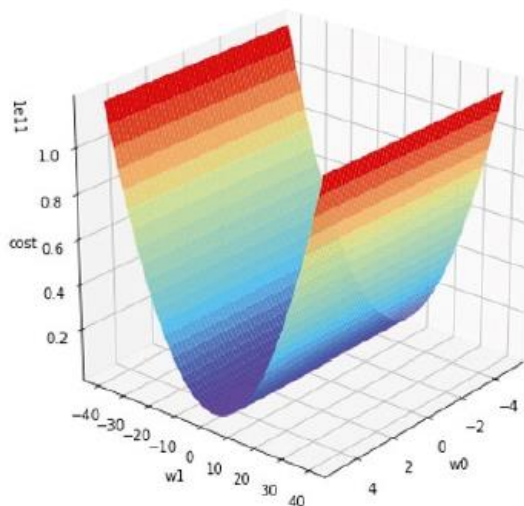
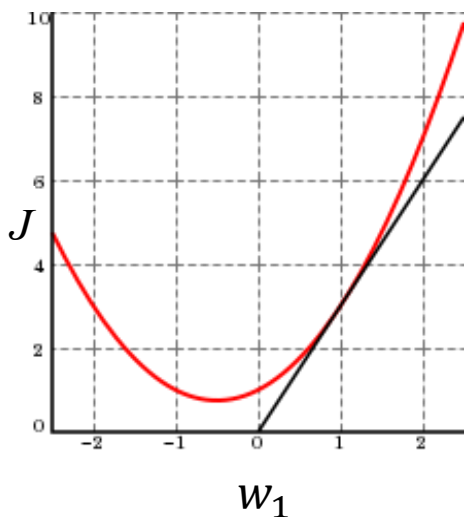
- 미분? 함수 그래프의 한 점에서의 접선의 기울기를 구하는 것
- 미분한 값을 0으로 만드는 최적값 w_i 을 구함





최소자승법의 활용

- $X^T X$ 가 존재할 때 문제의 해를 구할 수 있음
- 데이터의 개수가 피쳐의 개수보다 많은 경우가 대부분이라서 자주 사용됨
- 장점 : Iteration과 사용자가 지정하는 하이퍼 매개변수가 존재하지 않아서 데이터만 있으면 쉽게 해를 구할 수 있음
- 단점 : 피쳐가 늘어나면 속도가 느려짐
 - 현재 컴퓨터의 연산속도로는 다른 알고리즘에 비해 느린 것이 아님



Iteration: 가중치 업데이트



최소자승법 코드-데이터 준비

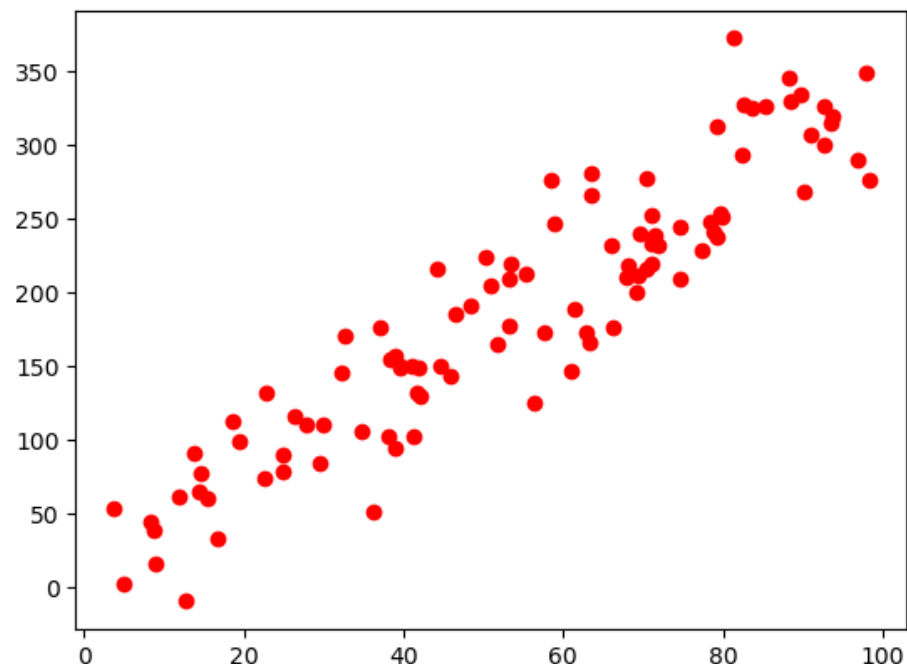
```
import matplotlib.pyplot as plt
import numpy as np

m = 100

X = np.random.uniform(0, 100, m)

# 직선 + 약간의 랜덤 노이즈
Y = 3*X + 30 + np.random.normal(0, 30, 100)

# 그래프
plt.scatter(X, Y, color="red")
plt.show()
```



100개

x																		
y																		



최소자승법 코드-모델 학습

```
from sklearn.linear_model import LinearRegression
```

```
#전처리: 구조 변경 행 벡터->열 벡터(2차원)
```

```
reX = X.reshape(-1,1)
```

```
# 모델 생성
```

```
model = LinearRegression()
```

```
# 학습
```

```
model.fit(reX, Y)
```

```
# 결과 확인
```

```
print("기울기 (coef):", model.coef_)
```

```
print("절편 (intercept):", model.intercept_)
```

```
기울기 (coef): [3.12042524]
```

```
절편 (intercept): 27.768832460147507
```

- 결과를 가설함수 식으로 표현하면 아래와 같음

$$h(\theta) = 3.1204 * x + 27.7688$$

p.15 y값 생성하는 코드

```
# 직선 + 약간의 랜덤 노이즈
```

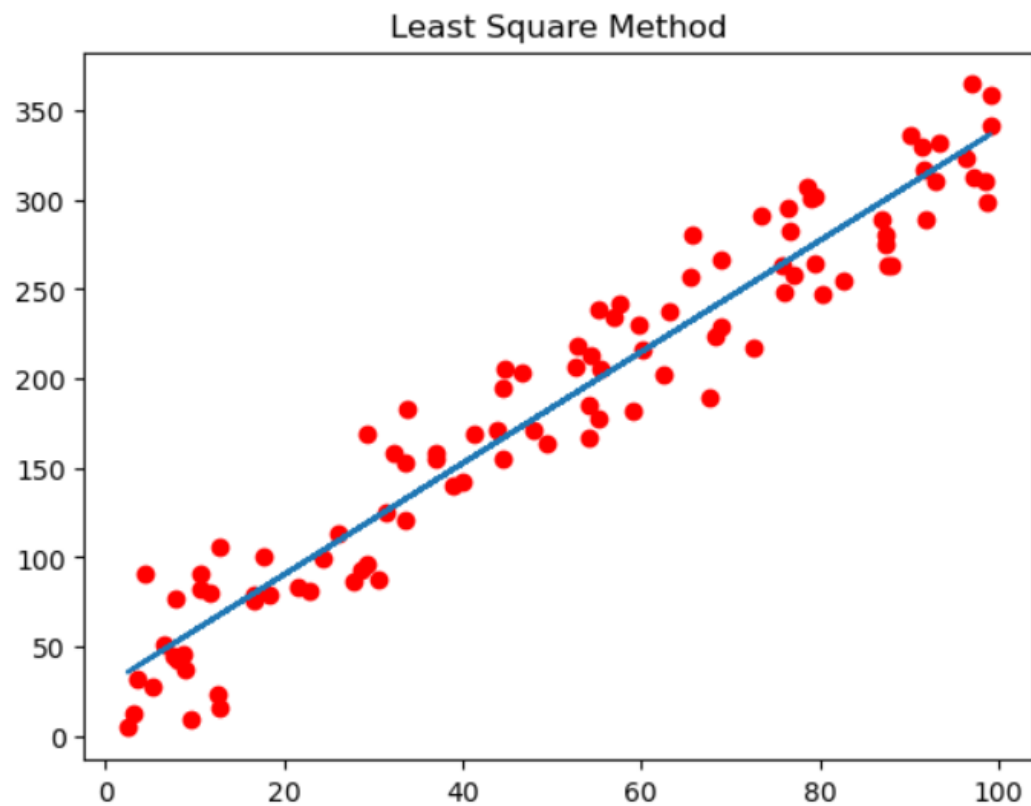
```
Y = 3*X+ 30 + np.random.normal(0, 30, 100)
```




최소자승법 코드 - 학습 결과

```
y_pred = model.predict(reX) # 예측
```

```
plt.title("Least Square Method")  
plt.scatter(x, y, color="red")  
plt.plot(x, y_pred)  
plt.show()
```



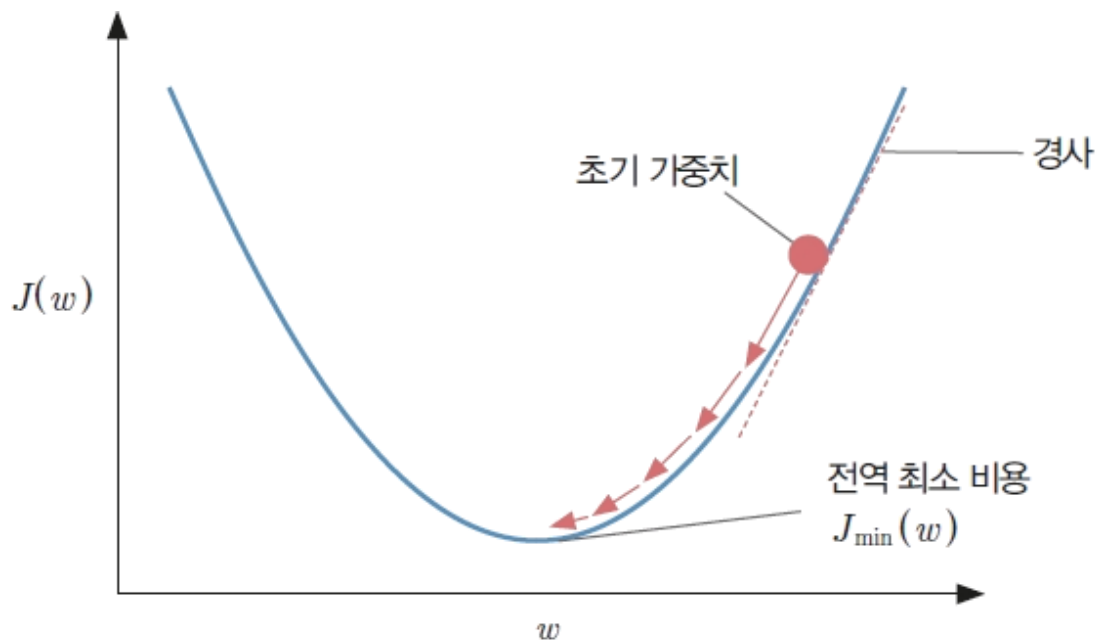
04

경사하강법으로 선형회귀 풀기



경사하강법의 개념

- 경사하강법(gradient descent) : 경사를 하강하면서 수식을 최소화하는 매개변수의 값을 찾아내는 방법
- 점이 최솟값을 달성하는 방향으로 점점 내려감
 - 몇 번 적용할 것인가? : 많이 실행할수록 최솟값에 가까워짐
 - 한 번에 얼마나 많이 내려갈 것인가?
 - 한 번에 얼마나 많은 공간을 움직일지를 기울기, 즉 경사(gradient)라고 부름





경사하강법 알고리즘

- $f(x)$ 는 최소화시켜야 하는 값이고 $2x$ 는 이를 미분한 값인 경사

$$f(x) = x^2 \rightarrow \frac{dy}{dx} = 2x$$

- 경사하강법의 기본 수식

$$x_{new} = x_{old} - \alpha \times (2x_{old})$$

경사값

- x_{old} 는 현재의 x 값, x_{new} 는 경사 값이 적용된 후 생성된 값
- 경사만큼의 변화가 계속 x 에 적용되어 x 의 최솟값 찾음
 - 반복적으로 미분 값을 적용시키면서 더 이상 값이 변하지 않거나 변화가 미미해지는 지점까지 값이 줄어듦

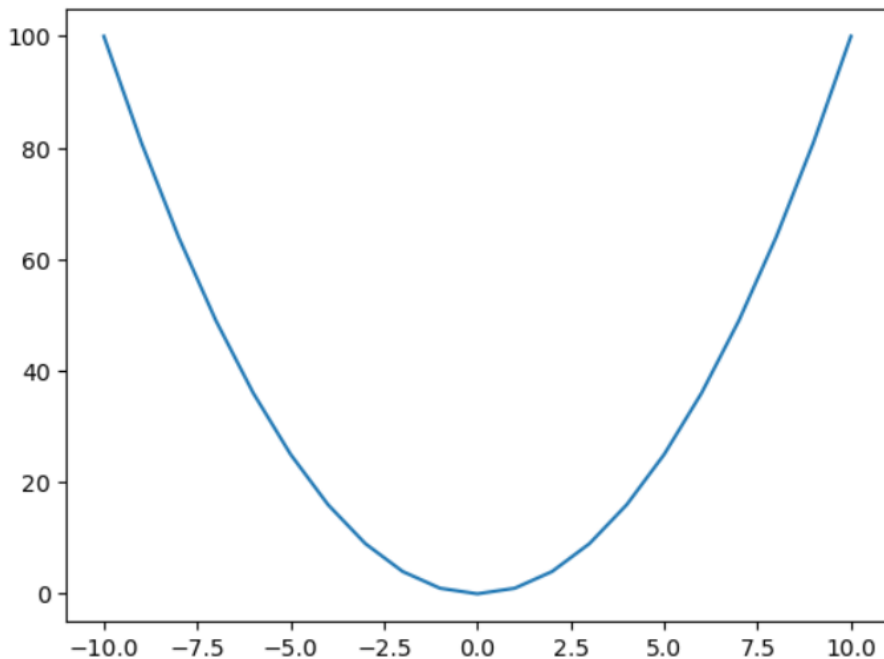


경사하강법 코드

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x= np.arange(-10,11,1)
f_x = x ** 2
```

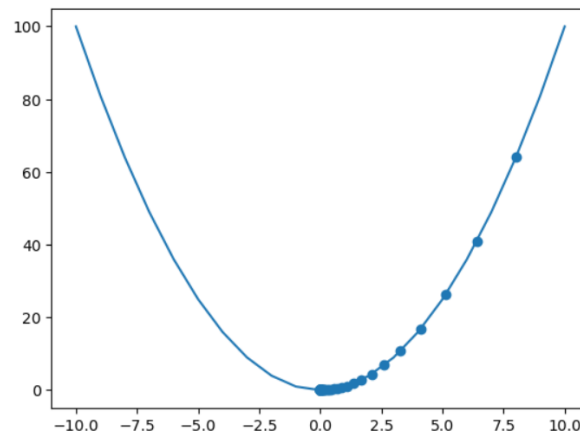
```
plt.plot(x, f_x)
plt.show()
```



```
x_new = 10
dotX = []
y = []
learng_rate= 0.1#gradient
```

```
for i in range(100):
    old_value = x_new
    dotX.append(old_value - learng_rate * 2 * old_value)
    x_new = old_value - learng_rate * 2 * old_value
    y.append(x_new ** 2)
```

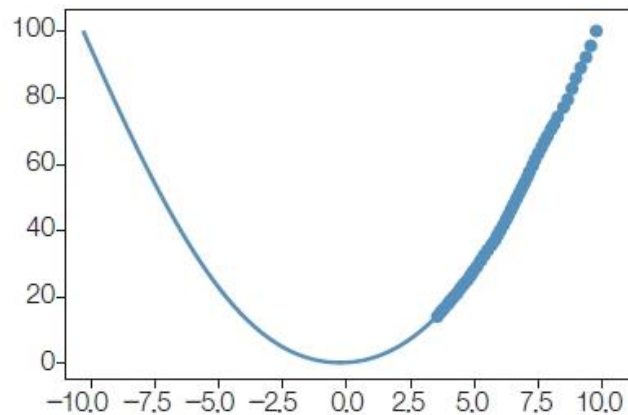
```
plt.plot(x, f_x)
plt.scatter(dotX, y)
plt.show()
```



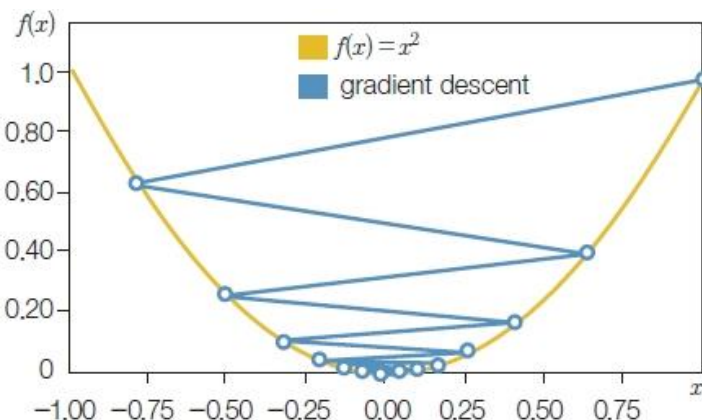


경사하강법에서 개발자가 결정해야 할 것

- 학습률(learning rate)을 얼마로 할 것인가? α 값을 결정 $x_{new} = x_{old} - \alpha \times (2x_{old})$
 - iteration이 수행될 때마다 최솟값 변화
 - 값이 너무 작으면 충분히 많은 iteration을 적용해도 원하는 최적값을 찾지 못하는 경우 발생
 - 값이 너무 크면 발산하여 최솟값 수렴 않거나 시간이 너무 오래 걸림



(a) 학습률이 작아 최솟값을 수렴하지 못하는 경우



(b) 학습률이 커서 발산한 경우

- 얼마나 많은 iteration으로 돌릴 것인가?
 - iteration 횟수가 충분하지 않다면 최솟값을 찾지 못하는 경우 발생
 - iteration 횟수가 너무 많다면 필요 없는 시간을 허비할 수도 있음



경사하강법으로 선형회귀 풀기

- J 를 최소화하는 방향으로 학습을 실행하고 그 때의 w_0 와 w_1 을 찾기
- w 변수가 두 개이기 때문에 3차원 그래프로 J 를 표현

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})^2$$

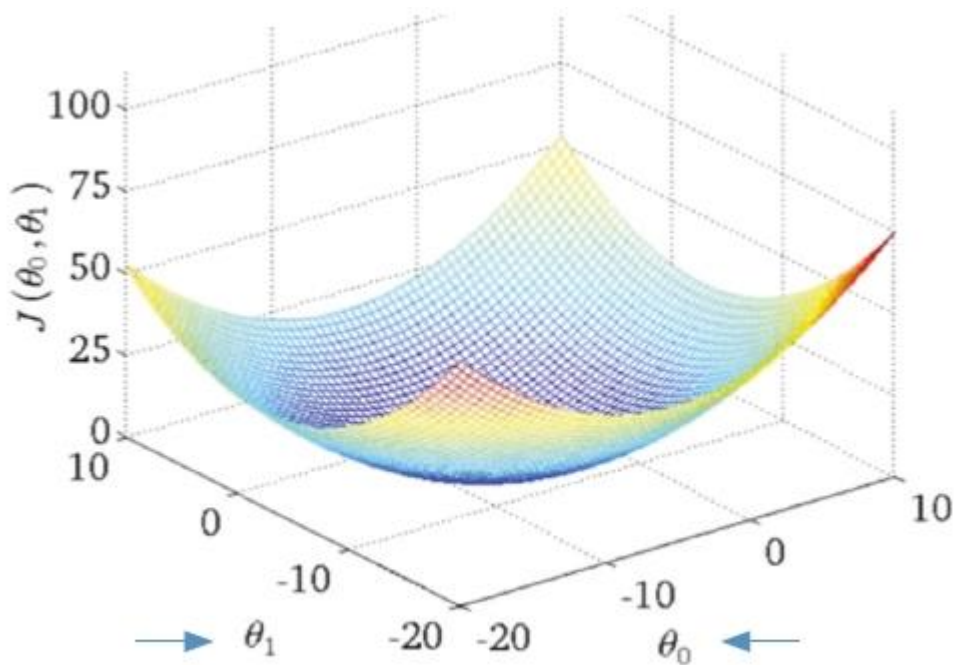


그림 7-9 비용함수 J 그래프 © Andrew Ng



가중치 w_n 에 대한 업데이트

- 가중치 w_0 와 w_1 에 대한 업데이트

- 먼저 임의의 w_0 와 w_1 을 지정
- 비용함수 J 가 최소화 될 때까지 iteration을 수행시켜 w_0 와 w_1 을 학습시킴
- 비용함수가 더 이상 줄어들지 않거나 정해진 iteration이 종료되면 학습이 끝남

loop until convergence {
do $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
}

$$\frac{\partial J}{\partial w_0} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})$$

$$\frac{\partial J}{\partial w_1} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)}) x^{(i)}$$

- w_0 와 w_1 업데이트는 동시에 일어나지만 하나의 iteration 안에서는 서로의 값에 영향을 주지 않음

05 선형회귀 성능 평가



데이터 형태에 따른 모델 성능

- 모델이 데이터에 과다적합(over-fit)된 경우 :
생성된 모델이 특정 데이터에만 잘 맞아서 해당 데이터셋에 대해서는 성능을 발휘할 수 있지만 새로운 데이터셋에서는 전혀 성능을 낼 수 없음
- 모델이 데이터에 과소적합(under-fit)된 경우 :
기존 학습 데이터를 제대로 예측하지 못함

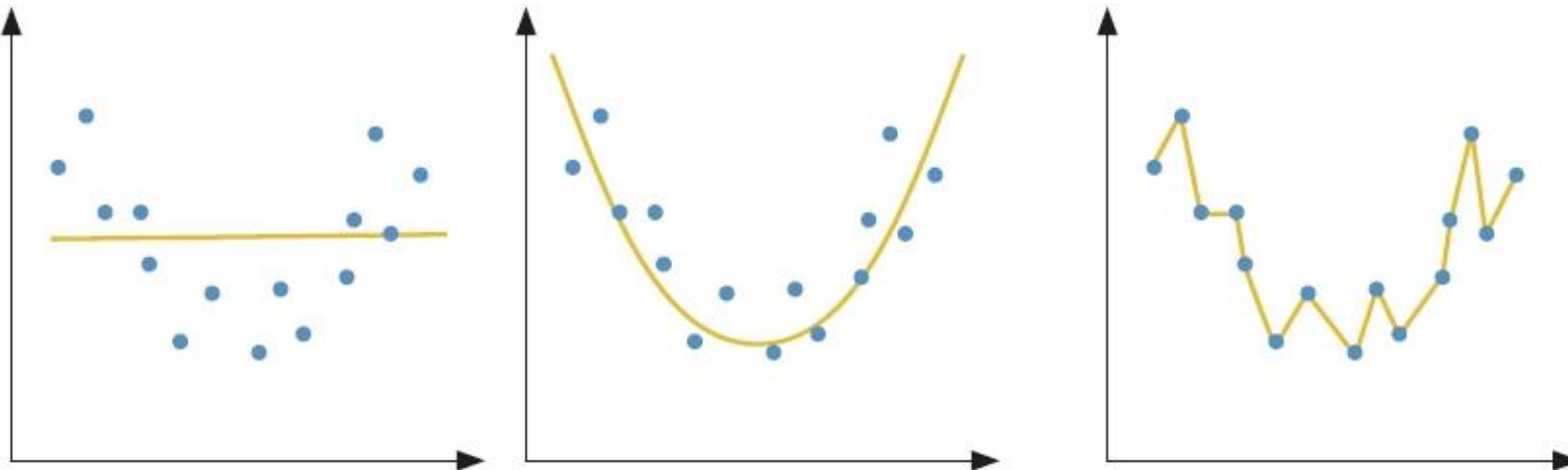


그림 7-10 다양한 데이터셋과 성능 파악



훈련/테스트 데이터 분할

- 훈련/테스트 분할(train/test split) : 머신러닝에서 데이터를 학습을 하기 위한 학습 데이터셋(train dataset) 과 학습의 결과로 생성된 모델의 성능을 평가하기 위한 테스트 데이터셋(test dataset)으로 나눔
- 모델이 새로운 데이터셋에도 일반화(generalize)하여 처리할 수 있는지를 확인
- 홀드아웃 메서드(hold-out method) : 전체 데이터셋에서 일부를 학습 데이터와 테스트 데이터로 나누는 일반적인 데이터 분할 기법
 - 전체 데이터에서 랜덤하게 학습 데이터셋과 테스트 데이터셋을 나눔
 - 일반적으로 7:3 또는 8:2 정도의 비율

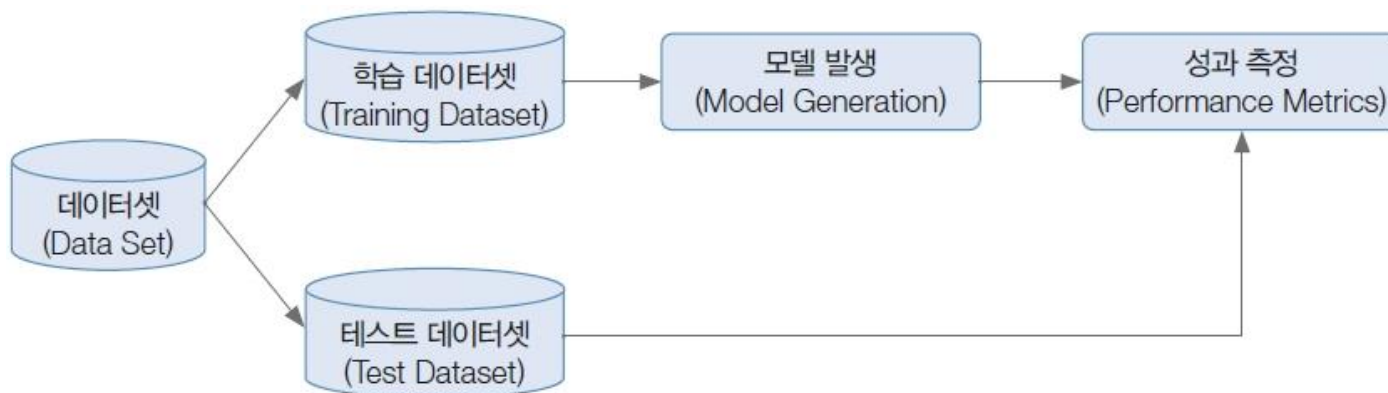


그림 7-11 홀드아웃 메서드(hold-out method) 기법



훈련/테스트 데이터 분할

- sklearn 모듈이 제공하는 train_test_split 함수 사용
 - X와 y 벡터 값을 각각 넣음
 - 매개변수 test_size에 테스트 데이터로 사용할 데이터의 비율을 지정
 - random_state는 랜덤한 값을 기준으로 임의로 지정하는 값

```
import numpy as np
from sklearn.model_selection import train_test_split

x, y = np.arange(10).reshape((5, 2)), range(5)

X_train, X_test, y_train, y_test = train_test_split(x,y, test_size=0.33, random_state=42)

print(X.shape)
print(len(y))
print('-'*15)
print(X_train.shape)
print(X_test.shape)
print(len(y_train))
print(len(y_test))
```



(5, 2)
5

(3, 2)
(2, 2)
3
2



선형회귀의 성능 평가 지표: MSE

- MSE(Mean Squared Error) : 평균 제곱 오차

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- 오차를 제곱하기 때문에 큰 오차에 더 큰 패널티를 주는 특징
 - 큰 오차(outlier)에 민감한 평가 지표
- 값이 0에 가까울수록 모델의 예측 성능이 좋음
- sklearn 모듈에서 mean_squared_error 함수로 MSE를 구함

```
from sklearn.metrics import mean_squared_error
```

```
y_true = [3, -0.5, 2, 7]
```

```
y_pred = [2.5, 0.0, 2, 8]
```

```
mean_squared_error(y_true, y_pred)
```

0.375



선형회귀의 성능 평가 지표: MAE

- MAE(Mean Absolute Error) : 실제값과 예측값의 절대 오차 평균
- 모든 테스트 데이터에 대해 예측값과 실제값의 차이에 대해 절댓값을 구하고 이 값을 모두 더한 후에 데이터의 개수만큼 나눈 결과

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| = \frac{1}{n} \sum_{i=1}^n |e_i|$$

- 직관적으로 예측값과 실측값의 차이를 알 수 있음
- sklearn 모듈에서는 mean_absolute_error 함수로 MAE를 구함

```
from sklearn.metrics import mean_absolute_error
```

```
y_true = [3, -0.5, 2, 7]
```

```
y_pred = [2.5, 0.0, 2, 8]
```

```
mean_absolute_error(y_true, y_pred)
```

0.5



선형회귀의 성능 평가 지표: RMSE

- RMSE(Root Mean Squared Error) : 평균 제곱근 오차
- 오차에 대해 제곱을 한 다음 모든 값을 더하여 평균을 낸 후 제곱근을 구함
- MAE에 비해 상대적으로 값의 차이가 더 큰 경우가 많음
- 차이가 크게 나는 값에 대해서 페널티를 주고 싶다면 RMSE 값을 사용

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- sklearn 모듈에서 root_mean_squared_error 함수로 RMSE를 구함

```
from sklearn.metrics import root_mean_squared_error
```

```
y_true = [3, -0.5, 2, 7]
```

```
y_pred = [2.5, 0.0, 2, 8]
```

```
root_mean_squared_error(y_true, y_pred)
```

0.6123724356957945

```
np.sqrt(mean_squared_error(y_true, y_pred))
```

0.6123724356957945



선형회귀의 성능 평가 지표: R-squared

- 결정계수(R-squared) : 모델이 데이터의 분산을 얼마나 잘 설명하는지를 나타내는 지표

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \mu)^2}$$

← 예측 오차 ← 실제 데이터의 전체 평균

값	의미
1	완벽한 예측
0	평균 예측과 동일
< 0	평균보다도 못한 모델

- 예측값이 크면 클수록 실제값도 커지고 예측값이 작으면 실제값도 작아짐
 - 데이터의 흠어짐을 모델이 얼마나 잘 맞추는지를 말함
 - 평균으로 예측하는 것보다 이 모델이 데이터를 얼마나 더 잘 맞추는가?

```
from sklearn.metrics import r2_score
```

```
y_true = [3, -0.5, 2, 7]
```

```
y_pred = [2.5, 0.0, 2, 8]
```

```
r2_score(y_true, y_pred)
```

```
0.9486081370449679
```


06

코드로 선형회귀 구현



데이터 전처리 및 초기 가중치 생성

```
#x 재생성
x = X.reshape(-1,1) # (100,) -> (100,1)
bias = np.ones((m,1))# bias column
x = np.hstack((x, bias))

#theta(가중치)생성
theta = np.random.uniform(0, 1, 2)

print(x[:5])
print("초기 가중치:", theta)
```

```
[[92.60749566  1.          ]
 [41.95994373  1.          ]
 [24.80182242  1.          ]
 [66.00639554  1.          ]
 [83.68868251  1.          ]]
초기 가중치: [0.07191804 0.65147994]
```

$$X = [x \quad 1] \quad \theta = \begin{bmatrix} 0.07 \\ 0.65 \end{bmatrix}$$
$$\hat{y} = X\theta$$



경사하강법으로 최적의 가중치 찾기

```
epoch = 50
alpha = 0.0005
theta_list = []
cost_list = []

for i in range(epoch):
    hypothesis = np.dot(x, theta)    # 예측값
    loss = hypothesis - y             # 오차
    cost = np.sum(loss ** 2) / (2 * m) # 비용함수
    gradient = np.dot(x.T, loss) / m  #  $\approx 0$ 
    theta = theta - alpha * gradient   # 가중치 업데이트

    # history 쌓기
    theta_list.append(theta)
    cost_list.append(cost)

print("최종 가중치:", theta)
```

최종 가중치: [3.47364728 0.58961101]

loop until convergence {
do $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
}

$$\frac{\partial J}{\partial w_0} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})$$
$$\frac{\partial J}{\partial w_1} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)}) x^{(i)}$$

행렬 미분 계산을 위해 x는 전치행렬 x.T 를 사용

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \quad X^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$



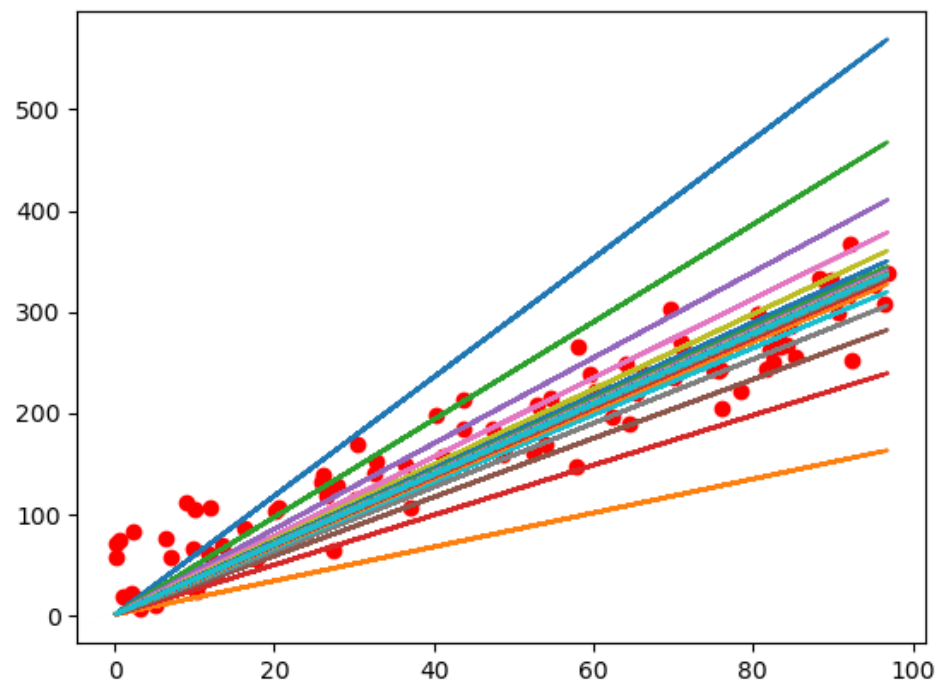
경사하강법 진행과정

```
y_hat_step = np.dot(x, np.array(theta_list).T)
```

```
plt.plot(x[:,0],y,"ro")#원본 데이터
```

```
for i in range (0, epoch):  
    plt.plot(x[:,0],y_hat_step[:,i])
```

```
plt.show()
```





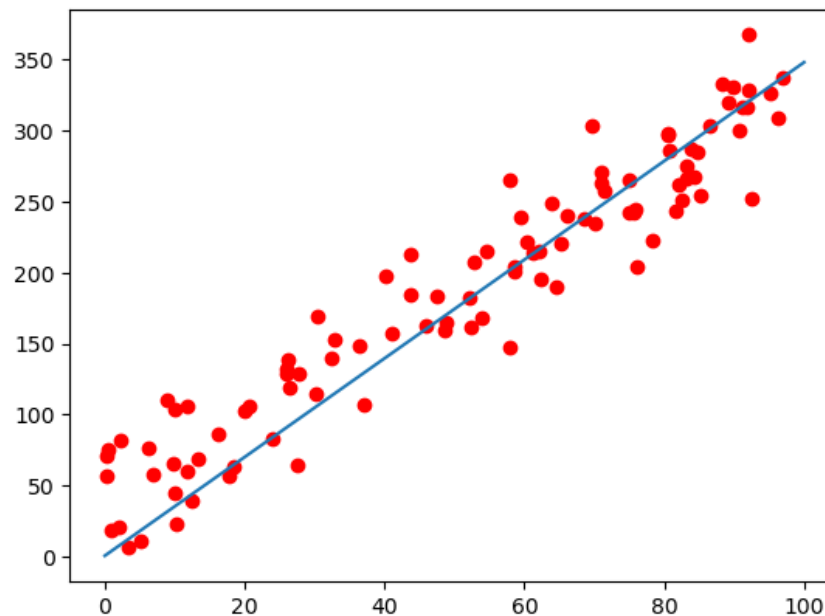
최종모델

```
#원본
plt.scatter(x[:,0],Y,color="red")

#최종 모델
def model(x):
    return theta[0]*x + theta[1]

temp = np.arange(0,100,1)
y_hat = model(temp)

plt.plot(temp,y_hat)
plt.show()
```



■ 최종 모델로 예측하기

```
data = 82 #새로운 데이터
print("data = ",data)
print("predict = ",model(data))
```



```
data = 82
predict = 285.4286880329367
```



성능 평가

#기존 데이터로 예측값

```
y_predict = model(x[:,0])
```

```
mse = mean_squared_error(Y, y_predict)
```

```
mae = mean_absolute_error(Y, y_predict)
```

```
rmse = root_mean_squared_error(Y, y_predict)
```

```
r2 = r2_score(Y, y_predict)
```

```
print("MSE:", mse)
```

```
print("MAE:", mae)
```

```
print("RMSE:", rmse)
```

```
print("R2:", r2)
```



MSE: 1014.6248391570922

MAE: 26.89501874223541

RMSE: 31.853176280507604

R2: 0.8785600309245128

- $R^2 = 0.8786 \rightarrow$ 비교적 높은 설명력
- $RMSE \approx 31.85 \rightarrow$ 평균 예측 오차 약 32 수준
 - 보통 $RMSE \geq MAE$
- 전체적으로 모델 성능이 비교적 양호한 편



SUMMARY

- 선형회귀: 독립변수와 종속변수 사이의 선형 관계를 이용하여 값을 예측하는 회귀 모델
- 가설함수: 입력 변수 x 를 이용하여 예측값 $\hat{y}$ 를 계산하는 선형회귀 모델
 - $\hat{y} = Xw$
- 비용함수: 예측값과 실제값 사이의 오차를 수치적으로 표현한 함수
- 비용함수를 최소화 하기 위한 방법
 - 최소자승법: 수학적 방법
 - 경사하강법: 비용함수의 기울기를 이용하여 반복적으로 가중치를 업데이트
- 회귀 모델의 성능 측정 지표
 - MSE (Mean Squared Error)
 - MAE (Mean Absolute Error)
 - RMSE (Root Mean Squared Error)
 - 결정계수 (R-squared)
- 경사하강법 코드 구현
 - 행렬 연산 $y = Xw$ 형태로 표현되며 경사하강법을 통해 최적의 가중치를 찾기 학습



Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일
dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝